IEEE Trans. on Robotics and Automation, 12(4), 566-580, 1996

Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces

Lydia E. Kavraki[1] Petr Švestka[2]
Jean-Claude Latombe[1] Mark H. Overmars[2]

Abstract

A new motion planning method for robots in static workspaces is presented. This method proceeds in two phases: a learning phase and a query phase. In the learning phase, a probabilistic roadmap is constructed and stored as a graph whose nodes correspond to collision-free configurations and whose edges correspond to feasible paths between these configurations. These paths are computed using a simple and fast local planner. In the query phase, any given start and goal configurations of the robot are connected to two nodes of the roadmap; the roadmap is then searched for a path joining these two nodes. The method is general and easy to implement. It can be applied to virtually any type of holonomic robot. It requires selecting certain parameters (e.g., the duration of the learning phase) whose values depend on the scene, that is the robot and its workspace. But these values turn out to be relatively easy to choose. Increased efficiency can also be achieved by tailoring some components of the method (e.g., the local planner) to the considered robots. In this paper the method is applied to planar articulated robots with many degrees of freedom. Experimental results show that path planning can be done in a fraction of a second on a contemporary workstation ($\approx$ 150 MIPS), after learning for relatively short periods of time (a few dozen seconds).

[1]Robotics Laboratory, Department of Computer Science, Stanford University, Stanford, CA 94305, USA.

[2]Department of Computer Science, Utrecht University, P.O.Box 80.089, 3508 TB Utrecht, The Netherlands.

Keywords: path planning, configuration space, probabilistic algorithms, many degrees of freedom robots, robotics.

1 Introduction

We present a new planning method which computes collision-free paths for robots of virtually any type moving among stationary obstacles (static workspaces). However, our method is particularly interesting for robots with many degrees of freedom (dof), say five or more. Indeed, an increasing number of practical problems involve such robots, while very few effective motion planning methods, if any, are available to solve them. The method proceeds in two phases: a *learning phase* and a *query phase*.

In the learning phase a probabilistic *roadmap* is constructed by repeatedly generating random free configurations of the robot and connecting these configurations using some simple, but very fast motion planner. We call this planner the *local planner*. The roadmap thus formed in the free configuration space (C-space [LP83]) of the robot is stored as an undirected graph R. The configurations are the nodes of R and the paths computed by the local planner are the edges of R. The learning phase is concluded by some postprocessing of R to improve its connectivity.

Following the learning phase, multiple queries can be answered. A *query* asks for a path between two free configurations of the robot. To process a query the method first attempts to find a path from the start and goal configurations to two nodes of the roadmap. Next, a graph search is done to find a sequence of edges connecting these nodes in the roadmap. Concatenation of the successive path segments transforms the sequence found into a feasible path for the robot.

Notice that the learning and the query phases do not have to be executed sequentially. Instead, they can be interwoven to adapt the size of the roadmap to difficulties encountered during the query phase, thus increasing the learning flavor of our method. For instance, a small roadmap could be first constructed; this roadmap could then be augmented (or reduced) using intermediate data generated while queries are being processed. This interesting possibility will not be explored in the paper, though it is particularly useful to conduct trial-and-error experiments in order to decide how much computation

time should be spent in the learning phase.

To run our planning method the values of several parameters must first be selected, e.g., the time to be spent in the learning phase. While these values depend on the *scene*, i.e., the robot and the workspace, it has been our experience that good results are obtained with values spanning rather large intervals. Thus, it is not difficult to choose one set of satisfactory values for a given scene or family of scenes, through some preliminary experiments. Moreover, increased efficiency can be achieved by tailoring several components of our method, in particular the local planner, to the considered robots. Overall, we found the method quite easy to implement and run. Many details can be engineered in one way or another to fit better the characteristics of an application domain.

We have demonstrated the power of our method by applying it to a number of difficult motion planning problems involving a variety of robots. In this paper we report in detail on experiments with planar articulated robots (or linkages) with many dofs moving in constrained workspaces. However, the method is directly applicable to other kinds of holonomic robots, such as spatial articulated robots in 3D workspaces [KL94b]. Additionally, a version of the method described here has been successfully applied to nonholonomic car-like robots [ŠO94]. In all cases, experimental results show that the learning times required for the construction of adequate roadmaps, i.e., roadmaps that capture well the connectivity of the free C-space, are short. They range from a few seconds[1] for relatively easy problems to a few minutes for the most difficult problems we have dealt with. Once a good roadmap has been constructed, planning queries are processed in a fraction of a second.

The very small query times make our planning method particularly suitable for many-dof robots performing several point-to-point motions in known static workspaces. Examples of tasks meeting these conditions include maintenance of cooling pipes in a nuclear plant, point-to-point welding in car assembly, and cleaning of airplane fuselages. In such tasks, many dofs are needed to achieve successive desired configurations of the end-effector while avoiding collisions of the rest of the arm with the complicated workspace. Explicit programming of such robots is tedious and time consuming. An efficient and reliable

[1] All running times reported in this paper have been obtained on a DEC Alpha workstation, except those given in Section 6 which were obtained with a Silicon Graphics Indigo workstation.

planner would considerably reduce the programming burden.

This paper is organized as follows. Section 2 gives an overview of some previous research and relates our work to this research. Section 3 describes our motion planning method in general terms, i.e., without focusing on any specific type of holonomic robot. Both the learning and the query phases are discussed here in detail. Next, in Sections 4, 5, and 6 we apply our method to planar articulated robots. In Section 4 we describe specific techniques that can be substituted for general ones in the planner to handle these robots more efficiently (especially when these have many dofs). In Sections 5 and 6 we describe a number of experiments and their results; we also analyze how variations of some parameter values affect planning results. Section 5 presents results obtained with a customized implementation of the method embedding the specific techniques of Section 4. Section 6 discusses other experimental results obtained with a general implementation of the method. Section 7 concludes the paper.

2 Relation to previous work

Path planning for robots in known and static workspaces has been studied extensively over the last two decades [Lat91]. Recently there has been renewed interest in developing heuristic, but practical path planners. For few-dof robots, many such planners have been designed and some are extremely fast (e.g., [BLL92, LRDG90]). Considerable attention is now directed toward the creation of efficient heuristic planners for many-dof robots. Indeed, while such robots are becoming increasingly useful in industrial applications, complete methods for such robots have overwhelming complexity. New emerging applications also motivate that trend, e.g., computer graphic animation, where motion planning can drastically reduce the amount of data input by human animators, and molecular biology, where motion planning can be used to compute motions of molecules (modeled as spatial linkages with many dofs) docking against other molecules.

The complexity of complete path planning methods in high-dimensional configuration spaces has led researchers to seek heuristic methods that embed weaker notions of completeness (e.g., probabilistic completeness) and/or can be partially adapted to specific problem domains in order to boost performance in those domains.

In recent years, some of the most impressive results were obtained using potential field methods. Such methods are attractive, since the heuristic function guiding the search for a path, the potential field, can easily be adapted to the specific problem to be solved, in particular the obstacles and the goal configuration. The main disadvantage of these planners is the presence of local minima in the potential fields. These minima may be difficult to escape. Local minima-free potential functions (also called navigation functions) have been defined in [Kod87, RK92, BL91]. But these functions are expensive to compute in high-dimensional configuration spaces and have not been used for many-dof robots.

One of the first successful potential field planners for robots with many dof is described in [FT87]. This planner has been used to compute paths of an 8-dof manipulator among vertical pipes in a nuclear plant, with interactive human assistance to escape local minima. In [FT90] the same authors present a learning scheme to avoid falling into the local minima of the potential field. During the learning phase, probabilities of moving between neighboring configurations without falling into a local minimum are accumulated in an r^n array, where n is the number of dofs and r is the number of intervals discretizing the range of each dof. During the planning phase, these probabilities are used as another heuristic function (in addition to the potential function) to guide the robot away from the local minima. This learning scheme was applied with some success to robots with up to 6 dofs. However, the size of the r^n array becomes impractical when n grows larger.

Techniques for both computing potential functions and escaping local minima in high-dimensional C-spaces are presented in [BL91, BLL92]. The Randomized Path Planner (RPP) described in [BL91] escapes local minima by executing random walks. It has been successfully experimented on difficult problems involving robots with 3 to 31 dofs. It has also been used in practice with good results to plan motions for performing riveting operations on plane fuselages [GMKL92], and to plan disassembly operations for the maintenance of aircraft engines [CL95]. Recently, RPP has been embedded in a larger "manipulation planner" to automatically animate scenes involving human figures modeled with 62 dofs [KKKL94]. However, several examples have also been identified where RPP behaves poorly [CG93, ZG93]. In these examples, RPP falls into local minima whose basins of attraction are mostly bounded by obstacles, with only narrow passages to escape. The probability that any random walk finds its way through such a passage is almost zero. In fact, once one knows how RPP computes the potential field, it is not too difficult to

create such examples. One way to prevent this from happening is to let RPP randomly use several potential functions, but this solution is rather time consuming. Our roadmap planner deals efficiently with problems that are difficult for RPP, as discussed in Section 5.

Other interesting lines of work include the method in [BF94] which is based on a variational dynamic programming approach and can tackle problems of similar complexity to the problems solved by RPP. In [GG92, GZ94] a sequential framework with backtracking is proposed for serial manipulators, and in [CH92] a motion planner with performance proportional to task difficulty is developed for arbitrary many-dof robots operating in cluttered environments. The planner in [Kon91] finds paths for six-dof manipulators using heuristic search techniques that limit the part of the C-space that is explored, and the planner in [ATBM92] utilizes genetic algorithms to help search for a path in high-dimensional C-spaces. Parallel processing techniques are investigated in [CG93, LPO91].

The planning method presented in this paper differs significantly from the methods referenced above, which are for the most part based on potential field or cell decomposition approaches. Instead, our method applies a roadmap approach [Lat91], that is, it constructs a network of paths in free C-space. Previous roadmap methods include the visibility graph [LPW79], Voronoi diagram [OY82], and silhouette [Can88] methods. All these three methods compute in a single shot a roadmap that completely represents the connectivity of the free C-space. The visibility graph and Voronoi diagram methods are limited to low-dimensional C-spaces. The silhouette method applies to C-spaces of any dimension, but its complexity makes it little practical.

Roadmaps have also been built and used incrementally in several other planners. The planner in [CL90] incrementally builds the skeleton of the C-space using a local opportunistic strategy. This work has inspired the approaches in [RC94, CB94] which construct retracts of the free C-space using sensor data and thus do not assume that the (static) environment in which the robot moves is known a priori. The approach in [Ch92] builds a sparse network of robot subgoals with the use of a simple and a computationally expensive planner. This network can also include information to accommodate local changes in the environment [Ch93, BCH94].

Our method emphasizes efficiency and is primarily developed for robots with many dofs which move in static environments. We are not aware of other roadmap techniques

that have been effectively applied to high-dimensional C-spaces. The approach we discuss in this paper uses probabilistic techniques to incrementally build a roadmap in the free C-space of the robot. It can produce a roadmap in any amount of allocated time. If the time spent on the construction of the roadmap is short, the roadmap may not adequately represent the connectivity of the free C-space. Actually, in our planner, the roadmap is never guaranteed to fully represent free C-space connectivity, though if we let our techniques run long enough it eventually will (but we don't know how long is enough). However, while building the roadmap, our method heuristically identifies "difficult" regions in free C-space and generates additional configurations in those regions to increase network connectivity. Therefore, the final distribution of configurations in the roadmap is not uniform across free C-space; it is denser in regions considered difficult by the heuristic function. This feature helps to construct roadmaps of a reasonable size that represent free C-space connectivity well. In particular, it allows our implemented planner to efficiently solve tricky problems requiring proper choices among several narrow passages, i.e., the kind of problems that potential field techniques like RPP tackle poorly.

Note also that, like most practical methods for many-dof robots (one exception is the method in [FT89]), RPP is a one-shot method, i.e., it does not precompute any knowledge of the free C-space that is transferred from one run to another. Consequently, on problems that both RPP and our method solve well, the latter is usually much faster, once it has constructed a good roadmap. But, if the learning time is included in the duration of the path planning process (which should be the case whenever planning is done only once in a given workspace), there are many problems for which RPP is faster.

The authors of this paper are from two different teams and the work presented here builds upon previous work they did separately. A single-shot random planner was described in [Ove92] and was subsequently expanded into a learning approach in [OŠ95]. In these papers the emphasis was on robots with a rather low number of dofs. Similar techniques have been applied both to car-like robots that can move forward and backward (symmetrical nonholonomic robots) and car-like robots that can only move forward [Šve93, ŠO94]. In [ŠO95] these results are extended to simultaneous motion planning for multiple car-like robots. Independently, a preprocessing scheme similar to the learning phase was introduced in [KL93]. This scheme also builds a probabilistic roadmap in free C-space, but focuses on the case of many-dof robots. The need to expand the roadmap in

"difficult" regions of C-space was noted there and addressed with simple techniques. Better expansion techniques were introduced in [KL94a, KL94b]. That approach is described in detail in [Kav95] and a theoretical analysis bounding the time spent by that planner is given in [KLMR95] and in [BKLLMR95]. The present paper combines the ideas of the experimental work in these previous papers. Since it only presents a limited subset of the experimental results we have obtained with our method, the interested reader is encouraged to look into our previous papers for additional results, in particular results involving other types of robots. Though computation times reported in these papers were obtained with different versions of our method, their orders of magnitude remain meaningful.

Finally, it should be noted that another planner which bares similarities with our approach, but was developed independently of our two teams, is proposed in [HST94].

3 The general method

We now describe our path planning method in general terms for a holonomic robot without focusing on any specific type of robot. During the learning phase a data structure called the roadmap is constructed in a probabilistic way for a given scene. The roadmap is an undirected graph $R = (N, E)$. The nodes in N are a set of configurations of the robot appropriately chosen over the free C-space. The edges in E correspond to (simple) paths; an edge (a, b) corresponds to a feasible path connecting the configurations a and b. These paths, which we refer to as local paths, are computed by an extremely fast, though not very powerful planner, called the local planner. The local paths are not explicitly stored in the roadmap, since recomputing them is very cheap. This saves considerable space, but requires the local planner to succeed and fail deterministically. We assume here that the learning phase is entirely performed before any path planning query. As we already noted, however, the learning and query phases can also be interwoven.

In the query phase, the roadmap is used to solve individual path planning problems in the input scene. Given a start configuration s and a goal configuration g, the method first tries to connect s and g to some two nodes $\tilde{s}$ and $\tilde{g}$ in N. If successful, it then searches R for a sequence of edges in E connecting $\tilde{s}$ to $\tilde{g}$. Finally, it transforms this sequence into a feasible path for the robot by recomputing the corresponding local paths

and concatenating them.

In the following, we let $\mathcal{C}$ denote the robot's C-space and $\mathcal{C}_f$ its free subset (also called the free C-space).

3.1 The learning phase

The learning phase consists of two successive steps, which we refer to as the construction and the expansion step. The objective of the former is to obtain a reasonably connected graph, with enough vertices to provide a rather uniform covering of free C-space and to make sure that most "difficult" regions in this space contain at least a few nodes. The second step is aimed at further improving the connectivity of this graph. It selects nodes of R which, according to some heuristic evaluator, lie in difficult regions of C-space and expands the graph by generating additional nodes in their neighborhoods. Hence, the covering of C_f by the final roadmap is not uniform, but depends on the local intricacy of the C-space.

3.1.1 The construction step

Initially the graph $R = (N, E)$ is empty. Then, repeatedly, a random free configuration is generated and added to N. For every such new node c, we select a number of nodes from the current N and try to connect c to each of them using the local planner. Whenever this planner succeeds to compute a feasible path between c and a selected node n, the edge (c, n) is added to E. The actual local path is not memorized.

The selection of the nodes to which we try to connect c is done as follows. First, a set N_c of candidate neighbors is chosen from N. This set is made of nodes within a certain distance of c, for some metric D. Then we pick nodes from N_c in order of increasing distance from c. We try to connect c to each of the selected nodes if it is not already graph-connected to c. Hence, no cycles can be created and the resulting graph is a forest, i.e., a collection of trees. Since a query would never succeed *thanks to* an edge that is part of a cycle, it is indeed sensible not to consume time and space computing and storing such an edge. However, in some cases, the absence of cycles may lead the query phase to construct unnecessary long paths. This drawback can easily be eliminated by applying

smoothing techniques to either the roadmap during the learning phase, or the particular paths constructed in the query phase, or both. Even if the roadmap contained cycles, such smoothing operations would eventually produce better paths.

Whenever the local planner succeeds in finding a path between two nodes, the connected components of R are dynamically updated. Therefore, no graph search is required for deciding whether a node picked from N_c is already connected to c, or not.

To make our presentation more precise, let:

- Δ be a symmetrical function $\mathcal{C}_f \times \mathcal{C}_f \rightarrow \{0,1\}$, which returns whether the local planner can compute a path between the two configurations given as arguments;
- D be a function $\mathcal{C} \times \mathcal{C} \rightarrow R^+ \cup \{0\}$, called the *distance function*, defining a pseudo-metric in $\mathcal{C}$. (We only require that D be symmetrical and non-degenerate.)

The construction step algorithm can be outlined as follows:

(1) $N \leftarrow \emptyset$
(2) $E \leftarrow \emptyset$
(3) **loop**
(4) $\quad c \leftarrow$ a randomly chosen free configuration
(5) $\quad N_c \leftarrow$ a set of candidate neighbors of c chosen from N
(6) $\quad N \leftarrow N \cup \{c\}$
(7) $\quad$ **forall** $n \in N_c$, in order of increasing $D(c,n)$ **do**
(8) $\quad\quad$ **if** $\neg$*same_connected_component*$(c,n) \wedge \Delta(c,n)$ **then**
(9) $\quad\quad\quad E \leftarrow E \cup \{(c,n)\}$
(10) $\quad\quad\quad$ update R's connected components

A number of components of algorithm above are still unspecified. In particular, we need to define how random configurations are created in (4), propose a local planner for (8), clarify the notion of a candidate neighbor in (5), and choose the distance function D used in (7).

Creation of random configurations. The nodes of R should constitute a rather uniform random sampling of $\mathcal{C}_f$. Every such configuration is obtained by drawing each of its

coordinates from the interval of values of the corresponding dof using the uniform probability distribution over this interval. The obtained configuration is checked for collision. If it is collision-free, it is added to N; otherwise, it is discarded.

Collision checking requires testing if any part of the robot intersects an obstacle and if two distinct bodies of the robot intersect each other. It can be done using a variety of existing general techniques. In the general implementation considered in Section 6 the test is performed analytically using optimized routines from the PLAGEO library [Gie93]. Alternatively, we could use an iterative collision checker, like the one described in [Qui94], which automatically generates successive approximations of the objects involved in the collision test. In 2D workspaces, we may use a faster, but more specific collision checker (see Section 4).

The local planner. Our best experimental results have been obtained when the local planner is both deterministic and very fast. These requirements are not strict, however. We discuss briefly the tradeoffs involved in the choice of the local planner.

If a non-deterministic planner was used, local paths would have to be stored in the roadmap. The roadmap would require more space, but this would not be a major problem.

Concerning how fast the local planner should be, there is clearly a tradeoff between the time spent in each individual call of this planner and the number of calls. If a powerful local planner was used, it would often succeed in finding a path when one exists. Hence, relatively few nodes would be required to build a roadmap capturing the connectivity of the free C-space sufficiently well to reliably answer path planning queries. Such a local planner would probably be rather slow, but this could be somewhat compensated by the small number of calls needed. On the other hand, a very fast planner is likely to be less successful. It will require more configurations to be included in the roadmap; so, it will be called more often, but each call will be cheaper.

The choice of the local planner also affects the query phase. The purpose of having a learning phase is to make it possible to answer path planning queries quasi-instantaneously. It is thus important to be able to connect any given start and goal configurations to the roadmap, or to detect that no such connection is possible, very quickly. This requires that the roadmap be dense enough, so that it always contains a few nodes (at least one)

to which it is easy to connect each of the start and goal configurations. It thus seems preferable to use a very fast local planner, even if it is not too powerful, and build large roadmaps with configurations widely distributed over the free C-space. In addition, if the local planner is very fast, we can use the same planner to connect the start and goal configurations to the roadmap at query time. Local paths needed not be memorized since recomputing them at query time is inexpensive. We actually tried several local planners, some very fast, some slower but more powerful, and our experimental observations clearly confirmed this conclusion (e.g., see [Mas92, Šve93]).

A quite general such local planner, which is applicable to all holonomic robots, connects any two given configurations by a straight line segment in configuration space and checks this line segment for collision and joint limits (if any). Verifying that a straight line segment remains within the joint limits is straightforward. On the other hand, collision checking can be done as follows [BL91]. First, discretize the line segment (more generally, any path generated by the local planner) into a number of configurations $c_1, \ldots, c_m$, such that for each pair of consecutive configurations (c_i, c_{i+1}) no point on the robot, when positioned at configuration c_i, lies further than some `eps` away from its position when the robot is at configuration c_{i+1} (`eps` is a predetermined positive constant).[2] Then, for each configuration c_i, test whether the robot, when positioned at c_i and "grown" by `eps`, is collision-free, using the collision checker discussed above. If none of the m configurations yield collision, conclude that the path is collision-free. Since `eps` is constant, the computation of the robot bodies grown by `eps` is done only once. In the following we will refer to this local planner as the *general local planner*.

The node neighbors. Another important choice to be made is that of the set N_c, the candidate neighbors of c. The local planner will be called to connect c with nodes in N_c and the cumulative cost of these invocations dominates learning time.

We avoid calls of the local planner that are likely to return failure by submitting only pairs of configurations whose relative distance (according to the distance function D) is smaller than some constant threshold `maxdist`. Thus, we define:

$$N_c = \{\tilde{c} \in N | D(c, \tilde{c}) \leq \texttt{maxdist}\}.$$

[2]Throughout this paper symbols in teletyped characters are used to denote parameters of the planning method.

Additionally, according to the algorithm outline given above, we try to connect c to all nodes in N_c in order of increasing distance from c; but we skip those nodes which are in the same connected component as c at the time the connection is to be tried. By considering elements of N_c in this order we expect to maximize the chances of quickly connecting c to other configurations and, consequently, reduce the number of calls to the local planner (since every successful connection results in merging two connected components into one).

In our experiments we found it useful to bound the size of the set N_c by some constant `maxneighbors` (typically on the order of 30). This additional criterion guarantees that, in the worst case, the running time of each iteration of the main loop of the construction step algorithm is independent of the current size of the roadmap R. Thus, the number of calls to the local method is linear in the size of the graph it constructs.

The distance function. The function D is used to both construct and sort the set N_c of candidate neighbors of each new node c. It should be defined so that, for any pair (c, n) of configurations, $D(c, n)$ reflects the chance that the local planner will *fail* to compute a feasible path between these configurations. One possibility is to define $D(c, n)$ as a measure (area/volume) of the workspace region swept by the robot when it moves along the path computed by the local planner between c and n in the absence of obstacles. Thus, each local planner would automatically induce its own specific distance function. Since exact computation of swept areas/volumes tends to be rather time-consuming, a rough but inexpensive measure of the swept-region gives better practical results. For example, when the general local planner described above is used to connect c and n, $D(c, n)$ may be defined as follows:

$$D(c, n) = \max_{x \in \text{robot}} \|x(n) - x(c)\|, \tag{1}$$

where x denotes a point on the robot, $x(c)$ is the position of x in the workspace when the robot is at configuration c, and $\|x(n) - x(c)\|$ is the Euclidean distance between $x(c)$ and $x(n)$.

3.1.2 The expansion step

If the number of nodes generated during the construction step is large enough, the set N gives a fairly uniform covering of the free C-space. In easy scenes R is then well connected.

But in more constrained ones where free C-space is actually connected, R often consists of a few large components and several small ones. It therefore does not effectively capture the connectivity of $\mathcal{C}_f$.

The expansion step is intended to improve the connectivity of the graph R generated by the construction step. Typically, if the graph is disconnected in a place where $\mathcal{C}_f$ is not, this place corresponds to some narrow, hence difficult region of the free C-space. The idea underlying the expansion step is to select a number of nodes from N which are likely to lie in such regions and to "expand" them. By expanding a configuration c, we mean selecting a new free configuration in the neighborhood of c, adding this configuration to N, and trying to connect it to other nodes of N in the same way as in the construction step. So, the expansion step increases the density of roadmap configurations in regions of $\mathcal{C}_f$ that are believed to be difficult. Since the "gaps" between components of the graph R are typically located in these regions, the connectivity of R is likely to increase.

We propose the following probabilistic scheme for the expansion step. With each node c in N we associate a positive weight $w(c)$ that is a heuristic measure of the "difficulty" of the region around c. Thus, $w(c)$ is large whenever c is considered to be in a difficult region. We normalize w so that all weights together (for all nodes in N) add up to one. Then, repeatedly, we select a node c from N with probability:

$$Pr(c \text{ is selected}) = w(c),$$

and we expand this node.

There are several ways to define the heuristic weight $w(c)$. One possibility is to count the number of nodes of N lying within some predefined distance of c. If this number is low, the obstacle region probably occupies a large subset of c's neighborhood. This suggests that $w(c)$ could be defined inversely proportional to the number of nodes within some distance of c. Another possibility is to look at the distance d_c from c to the nearest connected component not containing c. If this distance is small, then c lies in a region where two components failed to connect, which indicates that this region might be a difficult one (it may also be actually obstructed). This idea leads to defining $w(c)$ inversely proportional to d_c. Alternatively, rather than using the structure of R to identify difficult regions, we could define $w(c)$ according to the behavior of the local planner. For example, if the local planner often failed to connect c to other nodes, this is also an indication that

c lies in a difficult region. Which particular heuristic function should be used depends to some extent on the input scene. A more detailed discussion on expansion techniques can be found in [Kav95]. For the framework of this paper, the following function has produced good results:

- At the end of the construction step, for each node c, compute the failure ratio $r_f(c)$ defined by:

$$r_f(c) = \frac{f(c)}{n(c)+1},$$

  where $n(c)$ is total number of times the local planner tried to connect c to another node and $f(c)$ is the number of times it failed. (Note: Whenever the local planner fails to connect two nodes c and n, this failure is counted in *both* the failure ratios of c and n. In this way, the configurations that are included in N at the very beginning of the construction step get meaningful failure ratios.)

- At the beginning of the expansion step, for every node c in N compute $w(c)$ proportional to the failure ratio, but scaled appropriately so that all weights add up to one, i.e.:

$$w(c) = \frac{r_f(c)}{\sum_{a \in N} r_f(a)}.$$

To expand a node c, we compute a short random-bounce walk starting from c. For holonomic robots, a random-bounce walk consists of repeatedly picking at random a direction of motion in C-space and moving in this direction until an obstacle is hit. When a collision occurs, a new random direction is chosen. And so on. The final configuration n reached by the random-bounce walk and the edge (c, n) are inserted into R. Moreover, the path computed between c and n is explicitly stored, since it was generated by a non-deterministic technique. We also record the fact that n belongs to the same connected component as c. Then we try to connect n to the other connected components of the network in the same way as in the construction step. The expansion step thus never creates new components in R. At worst, it fails to reduce the number of components.

The weights $w(c)$ are computed only once at the beginning of the expansion step and are not modified when new nodes are added to R. Once the expansion step is over, all remaining small components of R, if any, are discarded. Here, a component is considered small if its number of nodes is less than some `mincomponent` percent (typically 0.01%)

of the total number of nodes in N. The graph R after discarding the small components represents the roadmap that will be used during the query phase. It may contain one or several components.

Let $\mathtt{T_L}$ be the time allocated to the learning phase. Clearly, the range of adequate values for $\mathtt{T_L}$ depends on the scene, and these value should be determined experimentally for each new scene. If $\mathtt{T_C}$ is the time spent on the construction step and $\mathtt{T_E}$ is the time spent on the expansion step, we have found that a 2:1 ratio between these times, i.e, $\mathtt{T_C} = 2\mathtt{T_L}/3$ and $\mathtt{T_E} = \mathtt{T_L}/3$, gives good results over a large range of problems.

3.2 The query phase

During the query phase, paths are to be found between arbitrary input start and goal configurations, using the roadmap constructed in the learning phase. Assume for the moment that the free C-space is connected and that the roadmap consists of a single connected component R. Given a start configuration s and goal configuration g, we try to connect s and g to some two nodes of R, respectively $\tilde{s}$ and $\tilde{g}$, with feasible paths P_s and P_g. If this fails, the query fails. Otherwise, we compute a path P in R connecting $\tilde{s}$ to $\tilde{g}$. A feasible path from s to g is eventually constructed by concatenating P_s, the recomputed path corresponding to P, and P_g reversed. If one wishes, this path may be improved by running a smoothing algorithm on it. Smoothing techniques that can be used here include the one in [LTJ90], which selects random segments of the global path and tries to shortcut them by using the local planner, and the method in [BG94], which iteratively performs local geometric operations (i.e., cutting off triangle corners).

The main question is how to compute the paths P_s and P_g. The queries should preferably terminate quasi-instantaneously, so no expensive algorithm is desired here. Our strategy for connecting s to R is to consider the nodes in R in order of increasing distance from s (according to D) and try to connect s to each of them with the local planner, until one connection succeeds. We ignore nodes located further than `maxdist` away from s, because the chance of success of the local planner is very low. If all connection attempts fail, we perform one or more random-bounce walks, as described in Subsection 3.1.2. But, instead of adding the node at the end of each such random-bounce walk to the roadmap, we now try to connect it to R with the local planner. As soon as s is successfully connected

to R, we apply the same procedure to connect g to R.

The reconstruction of a robot path from the sequence of nodes in P reduces to the concatenation of the paths that take the robot between adjacent nodes in P. Some of these paths have been produced by random-bounce walks during the learning phase and are stored in the relevant edges of R. Paths that correspond to connections that have been found during learning by the local planner are recomputed. The local planner is deterministic and it will produce the same path every time it is called with the same input configurations. Collisions need not be checked along the recomputed local paths if the local planner has the property that it aborts when a collision is detected: all intermediate configurations along the path have been checked for collision when the local path was first computed. An example of a planner having the above property is the straight-line planner of Section 3.1.1. If the local planner performs a certain (deterministic) action when a collision is detected, then collisions need to be checked along the recomputed path so that the same action can be repeated just after a collision is detected.

In general, however, the roadmap may consist of several connected components R_i, $i = 1, 2, \ldots, p$. This is usually the case when the free C-space is itself not connected. It may also happen when free C-space is connected, for instance if the roadmap is not dense enough. If the roadmap contains several components, we try to connect both s and g to two nodes in the *same* component, starting with the component closest to s and g. If the connection of s and g to some component R_i succeeds, a path is constructed as in the single-component case. The method returns failure if it cannot connect both the start and goal configuration to the same roadmap component. Since in most examples the roadmap consists of rather few components, failure is rapidly detected.

If path planning queries fail frequently, this is an indication that the roadmap may not adequately capture the connectivity of the free C-space. Hence, more time should be spent in the learning phase, i.e., $\mathtt{T_L}$ should be increased. However, it is not necessary to construct a new roadmap from the beginning. Since the learning phase is incremental, we can simply extend the current roadmap by resuming the construction step algorithm and/or the expansion step algorithm, starting with the current roadmap graph, thus interweaving the learning and the query phases.

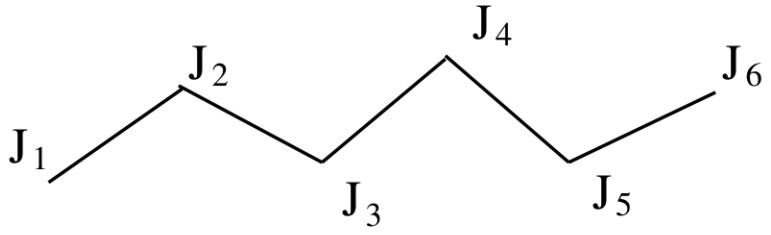


Figure 1: A planar articulated robot.

4 Application to planar articulated robots

This section describes the application of our planning method to planar articulated robots with fixed or free bases. We present techniques specific to these robots that can be substituted for the more general techniques in the roadmap method in order to increase its efficiency. The purpose of this presentation is to illustrate the ease with which the general method for holonomic robots can be engineered to better suit the needs of a particular application. In Section 5 we will discuss experiments with an implementation of the method that embeds the specific techniques described below, while in Section 6 we will present experimental results with a general implementation of the method to demonstrate that the method remains quite powerful, even without specific components. In the rest of the paper we will refer to these two implementations as the *customized implementation* and the *general implementation*, respectively.

To make the following presentation shorter, we only consider the following type of planar articulated robots with an arbitrary number of revolute joints. Figure 1 illustrates such a robot in which the links are line segments. The links, which may actually be any polygons, are denoted by L_1 through L_q (in the figure, $q = 5$). Points J_1 through J_q designate revolute joints. Point J_1 denotes the base of the robot; it may, or may not, be fixed relative to the workspace. The point J_{q+1} (J_6 in the figure) is called the endpoint of the robot. Each revolute joint J_i ($i = 1, \ldots, q$) has defined certain internal joint limits, denoted by low_i and up_i, with $low_i < up_i$, which constrain the range of the possible orientations that L_i can take relative to L_{i-1}. If the robot's base is free, the translation of J_1 is bounded along the x and y axes of the Cartesian coordinate system embedded

in the workspace by low_x and up_x, and low_y and up_y, respectively. We represent the C-space of such a q-link planar articulated robot by $[low_1, up_1] \times \ldots \times [low_q, up_q]$, if its base is fixed, and by $[low_x, up_x] \times [low_y, up_y] \times [0, 2\pi] \times [low_2, up_2] \times \ldots \times [low_q, up_q]$, if its base is free. A self-collision configuration is any configuration where two non-adjacent links of the robot intersect each other. We do not allow such configurations. Thus, the free C-space is constrained by the obstacles and by the set of self-collision configurations. We assume that the joint limits prevent self-collisions between any two adjacent links.

We now discuss specific techniques for local path planning, distance computation, and collision checking that apply well to the family of robots defined above. The same techniques can also be applied, possibly with minor adaptations, to other types of articulated robots, e.g., robots with prismatic joints and/or with multiple kinematic chains [KL94a] and articulated robots in 3D workspace [KL94b].

Local path planning. Let a and b be any two given configurations that we wish to connect with the local planner. Our local planner constructs a path as follows. It translates at constant relative velocity all the joints with an odd index, i.e., all J_{2*i+1}'s, along the straight lines in the workspace that connect their positions at configuration a to their positions at configuration b. During this motion the planner adjusts the position of every other joint J_{2*i} using the straightforward inverse kinematic equations of this point relative to J_{2*i-1} and J_{2*i+1}. Thus, the J_{2*i}'s "follow" the motion led by the J_{2*i+1}'s. If q is even, the position of J_q is not determined by the above rule; it is computed by rotating joint J_q at constant revolute velocity relative to the linear velocity of point J_q. Recall from Subsection 3.1.1 that a local path is discretized into a sequence of configurations for collision checking. When our specific technique is used, we must also verify that the coordinates of each such configuration are within joint limits. Thus, the motion is aborted if either a collision occurs, or a joint moves beyond one of its limits, or some J_{2*i} cannot follow the motion led by the J_{2*i+1}'s. We have observed that in cases when the above motion does not manage to connect configurations a and b, it nevertheless brings the robot to a configuration b' very close to b. It then pays off to try to connect b' and b with a straight line in C-space and only after this fails to declare failure of the local planner to connect a and b. We will refer to the above planner as the *specific local planner*.

The workspace region swept out by the robot along a local path computed by the

specific local planner between two configurations a and b is typically smaller than for the path joining a and b by a straight line segment in configuration space, which is computed by the general local planner described in Subsection 3.1.1. Hence, the local paths generated by the specific planner are more likely to be collision-free than those generated by the general planner. Also, collision checking is less expensive since, for a given `eps`, the discretization of the local path yields less configurations. On the other hand, the specific planner, though still very fast, is not as fast as the general planner. Indeed, it requires inverse kinematic computation to determine configuration coordinates along the path. Nevertheless, on examples involving many-dof planar articulated robots, better results are obtained when the specific local planner is used.

Distance computation. Let $J_i(a)$, $i = 1, \ldots, q+1$ denote the position of the point J_i in the workspace, when the robot is at configuration a. We define the distance function D by:

$$(a, b) \in \mathcal{C} \times \mathcal{C} \quad \mapsto \quad D(a, b) = \sum_{i=1}^{q+1} \|J_i(a) - J_i(b)\|^2,$$

where $\|J_i(x) - J_i(y)\|$ is the Euclidean distance between $J_i(a)$ and $J_i(b)$. This function is a better approximation of the area swept by the robot along the local paths computed by the specific local planner than the general distance function defined by Eqn. (1).

Collision checking. The 2D workspace allows for a very fast collision checking technique. In this technique each link of the robot is regarded as a distinct robot with two dofs of translation and one dof of rotation. A bitmap representing the 3D configuration space of this robot is precomputed, with the "0"'s describing the free subset of this space and the "1"'s describing the subset where the link collides with an obstacle. When a configuration is checked for collision, the 3D configuration of each link is computed and tested against its C-space bitmap, which is a constant-time operation. Different 3D bitmaps must be computed for links of different shape. However, if larger links can be modeled as two (or more) smaller links, then we need not create one bitmap for each link of the robot. For example, when all the links are line segments (as in Figure 1), a single bitmap can be computed, for the shortest link. Then collision checking for a long link requires multiple access to the bitmap of the short link. The computation of any 3D bitmaps needed for

collision checking is performed only once, prior to the learning phase.

The 3D bitmap for one link can be computed as a collection of 2D bitmaps, each corresponding to a fixed orientation of the link. If the link and the obstacles are modeled as collections of possibly overlapping convex polygons, the construction of a 2D bitmap can be done as follows [LRDG90]. First use the algorithm in [LP83] to produce the vertices of the obstacles in the link's C-space. (This algorithm takes linear time in the number of vertices of the objects.) Then draw and fill the obstacles into the 2D bitmap. (On many workstations, this second operation can be done very quickly using raster-scan hardware originally designed to efficiently display filled polygons on graphic terminals.) Each 2D bitmap may also be computed using the FFT-based method described in [Kav93], whose complexity depends only on the size of the bitmap. This FFT method is advantageous when the obstacles are originally input as bitmaps. In any case, experiments show that computing a 3D bitmap with a size on the order of $128 \times 128 \times 128$ takes a few seconds. Clearly, this technique is not yet practical for 3D workspaces, since it requires the generation of 6D bitmaps.

There are many other ways of adjusting our general path planning method to a specific robot. However, too much specific tuning may not always be desirable: at some point the gain in efficiency becomes smaller than the burden of making the specific changes and keeping track of them.

5 Results with customized implementation

In this section we present the performance of our method when this is implemented with the local planner, the collision checker, and the distance function described in Section 4. To be precise, while collision checking with obstacles is done using the bitmap technique, self-collisions are detected analytically.

The planner is implemented in C and for the experiments reported here we used a DEC Alpha workstation. This machine is rated at 126.0 SPECfp92 and 74.3 SPECint92. We have tested our planner on a number of 2D scenes. Each scene contains polygonal obstacles and a planar articulated robot whose links are line segments (see Figures 2 and 6). By no means does this reflect a limitation of the method. The specific local

planner and collision checker of Section 4 also apply to robots made of polygonal links (though several bitmaps may then be required). The parameters of our planner are:

- $\mathtt{T_C}$, the time to be spent in the construction step;
- $\mathtt{T_E}$, the time to be spent in the expansion step;
- `maxdist`, the maximal distance between nodes that the local planner may try to connect;
- `eps`, the constant used to discretize local paths before collision checking;
- `maxneighbors`, the maximum number of calls of the local planner per node;
- $\mathtt{T_{RB_expand}}$, the duration of the computation of a random-bounce walk performed during the expansion step (learning phase);
- $\mathtt{N_{RB_query}}$, the maximum number of random-bounce walks allowed for connecting the start or goal configuration to the roadmap (query phase);
- $\mathtt{T_{RB_query}}$, the duration of the computation of each of the random-bounce walks during the query phase.

(Notice that the last two parameters determine an upper bound on the time spent in answering a query.)

For each scene, we first input a set of configurations by hand, which we refer to as the test set. For a fixed $\mathtt{T_C}$ and $\mathtt{T_E}$, we then independently create many different roadmaps starting with different values of the random value generator. In the examples here we only keep the largest connected component of the roadmap; other components, if any, are simply discarded. We then try to connect the same configuration in the test set to each of these roadmaps and we record the percentage of times our planner succeeds to make a connection in a prespecified amount of time (2.5 seconds). The estimated success rates may be used to calculate the success rates of queries that involve any two configurations in the test set. By performing a large number of experiments, we believe that we present a realistic characterization of the performance of our planner. In particular, we ensure that the results do not reflect just a lucky run, or a bad one. We independently repeat the

same experiment for different $\mathtt{T_C}$ and $\mathtt{T_E}$. For the other parameters described above, we choose fixed values throughout the experiments based on some preliminary experimental results. Notice that it is important to choose the configurations in the test set manually. For obvious reasons, a random generation similar to the one used during the learning phase tends to produce configurations that are easily connected to the roadmap. Instead, proceeding manually allows us to select "interesting" configurations, for example configurations where the robot lies in narrow passages between workspace obstacles. It is unlikely that the random generator of the learning phase produced many such configurations.

We present results obtained with two representative scenes shown in Figures 2 (fixed-base robot) and 6 (free-base robot).

Fixed-base articulated robot. Figure 2 shows eight configurations forming the test set of an articulated robot in a scene with several narrow gates. The robot has a fixed base, denoted by a square, and 7 revolute degrees of freedom.

The table in Figure 3 reports the success rates of connecting the configurations in the test set to roadmaps obtained with different learning times. The learning time, $\mathtt{T_L}$, is shown in column 1. It is broken into $\mathtt{T_C}$ and $\mathtt{T_E}$ in columns 2 and 3, with $\mathtt{T_E} = \mathtt{T_C}/2$. The values of the other parameters of the planner are: $\mathtt{maxdist} = 0.4$, $\mathtt{eps} = 0.01$ (for the interpretation of these two values note that the workspace is described as a unit square), $\mathtt{maxneighbors} = 30$, $\mathtt{T_{RB_expand}} = 0.01$ sec, $\mathtt{T_{RB_query}} = 0.05$ sec, $\mathtt{N_{RB_query}} = 45$.

For every row of the table in Figure 3 we independently generated 30 roadmaps, each with the indicated learning time. The roadmaps generated for different rows were also computed independently, that is, no roadmap in some row was reused to construct a larger one in following row.

Column 4 in Figure 3 gives the average number of collision checks performed for the roadmap construction for different learning times. This number can be regarded as an estimate of the computational complexity of the planner. In the context of the our approach, it must be interpreted with caution. Collision checks are done not only along robot paths, as in most planners, but also when trying to guess random free configurations of the robot (see Subsection 3.1.1). Most of these randomly guessed configurations are illegal because of collisions with the obstacles or self-intersections. On the average, this

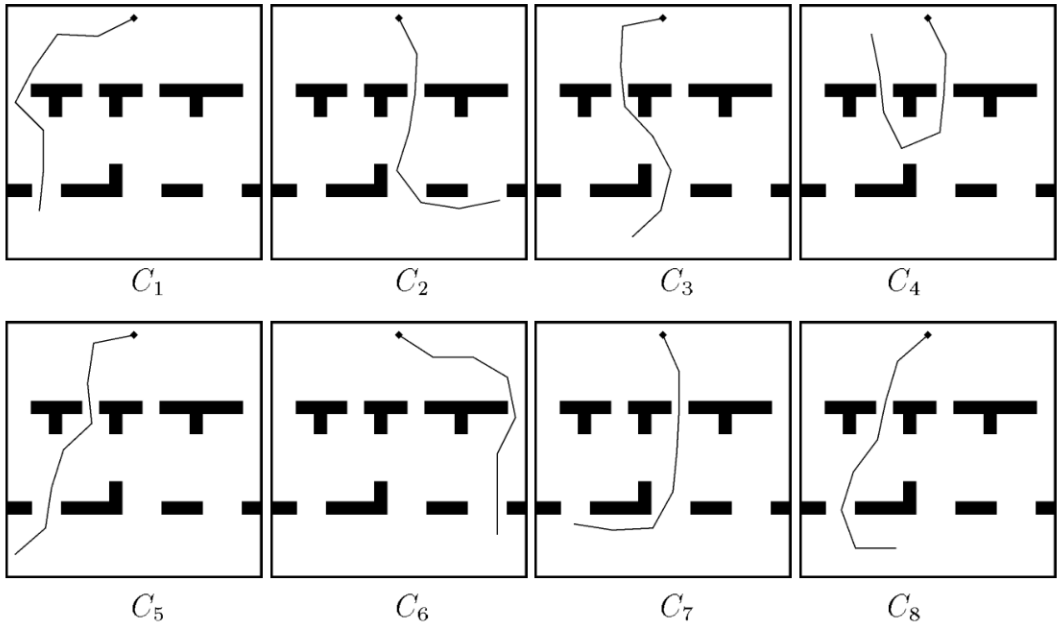


Figure 2: Scene 1, with 7-revolute-joint fixed-base robot.

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks	Avg. nodes	Success Rate (%) C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
20.1	13.1	7.0	621943	1062	100.0	36.7	56.7	36.7	53.3	100.0	36.7	60.0
30.1	19.5	10.6	889384	1643	100.0	66.7	70.0	66.7	76.7	100.0	66.7	80.0
40.3	26.3	14.0	1145091	2233	100.0	90.0	86.7	90.0	86.7	100.0	90.0	86.7
50.3	32.7	17.6	1392454	2783	100.0	96.7	96.7	96.7	96.7	100.0	96.7	96.7
60.2	39.1	21.1	1631612	3284	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
70.3	45.8	24.5	1876006	3805	100.0	96.7	100.0	96.7	100.0	100.0	96.7	100.0
80.4	52.2	28.2	2104209	4272	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Figure 3: Results with customized planner for scene of Fig. 2 (with expansion).

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks	Avg. nodes	Success Rate (%) C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
20.0	20.0	0.0	597559	1011	100.0	13.3	36.7	10.0	40.0	93.3	13.3	36.7
30.1	30.1	0.0	852038	1601	100.0	50.0	46.7	46.7	46.7	90.0	53.3	46.7
40.2	40.2	0.0	1086053	2300	100.0	80.0	80.0	80.0	80.0	100.0	80.0	80.0
50.2	50.2	0.0	1291216	2877	100.0	90.0	96.7	90.0	96.7	100.0	90.0	96.7
60.2	60.2	0.0	1502089	3372	100.0	90.0	100.0	90.0	100.0	100.0	90.0	100.0
70.2	70.2	0.0	1688544	3877	100.0	96.7	100.0	96.7	100.0	100.0	96.7	100.0
80.3	80.3	0.0	1860341	4295	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Figure 4: Results with customized planner for scene of Fig. 2 (no expansion).

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks	Size of components	Coll. checks for connection to roadmap							
					C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
20.3	13.3	7.0	620238	878, 116, 62	62	F	F	F	F	F	F	F
30.2	19.7	10.5	905312	1644, 165	78	51	F	7584	F	40	59	F
40.4	26.3	14.1	1178494	2411	53	1148	22	3432	33	44	225	2270
50.3	32.8	17.5	1421185	2881, 63, 10	13	20	20	3877	80	38	20	2328
60.4	39.3	21.1	1661916	3302, 35, 33	57	45	16	22	14	160	51	46
70.2	45.6	24.6	1917744	3869, 52, 10	94	30	19	4764	21	42	74	63
80.2	52.1	28.1	2128273	4245, 49	32	25	16	32	12	89	48	43

Figure 5: Connecting configurations to the roadmap.

is quickly detected and the collision checker aborts almost instantaneously. Column 5 in Figure 3 reports the average number of nodes, over the 30 runs, in the largest roadmap component at the end of the learning phase. The largest connected component of each roadmap is used for query processing. Columns 6 though 13 are labeled with the eight configurations $C_1, \ldots, C_8$ of Figure 2. The columns report the success rate when trying to connect, in less that 2.5 seconds, the corresponding configuration to each of the 30 produced roadmaps. One trial (as defined by the parameters `maxdist`, `maxneighbors`, $T_{\texttt{RB_query}}$, and $N_{\texttt{RB_query}}$) was made per roadmap.

The table in Figure 3 shows that after a learning time of 60 seconds or more (rows 5, 6, and 7), all eight configurations of Figure 2 are successfully connected to the generated roadmaps with very few exceptions. These are all located in row 6, where configurations C_3, C_4 and C_7 were not connected to the roadmaps, once out of the 30 trials of that row. Such exceptions are to be expected with a randomized technique.

Figure 4 shows the percentage of successful connections to roadmaps created without expansion. The corresponding rows of the tables in Figures 3 and 4 report results obtained with the same learning time. We again generated 30 independent roadmaps in each row in Figure 4. We show the average number of collision checks required to create each roadmap (column 4), the average number of nodes in the largest component of these roadmaps (column 5), and the success rate when trying to connect $C_1, \ldots, C_8$ to them. In

general, the percentages of successful connections are lower in this table. The difference shows more clearly when the learning time is small. If we are interested in obtaining a solution to a path planning problem as fast as possible, it is thus better to spend part of the time allocated to the learning phase on the expansion step rather than spend it completely on the construction step. As mentioned above, the ratio $\mathtt{T_C}/\mathtt{T_E} = 2$ gives good results over a wide range of problems.

Let us finally note that connecting $C_1, \ldots, C_8$ to the roadmaps is very fast, which in turn means that finding a path between any two of the above configurations is also fast. In Figure 5 we repeat the experiment of Figure 3, but now we create only one roadmap in each row of the table. We report in columns 6 to 13 the actual number of collision checks needed to connect $C_1, \ldots, C_8$ to the roadmaps produced after learning times of 20, 30, 40, 50, 60, 70 and 80 seconds. Again, we try to connect the configurations in the test set only to the largest component of these roadmaps, and we report failure (indicated by 'F') if we do not succeed to do so within the allocated time (2.5 seconds). In most cases, relatively few collision checks are needed for successful connection to the roadmap: a few tens to a few hundreds. Infrequently, a couple of thousands of collision checks are performed. This happens when one or more random-bounce walks are executed before the configuration is connected to the roadmap with the local planner. In any case, for the machine used for our experiments, the above numbers translate to connection times of a fraction of second to a few seconds. In the table of Figure 5 we report in column 4 the size of all roadmap components with more than 10 nodes. It is easy to see that after a learning time of 40 seconds, there is a clear difference in the size of the major component and the smaller ones. The latter contain only a small percentage of the total nodes and their presence does not affect path planning. That is why in our analysis we considered only the largest component of the roadmaps produced by learning.

Path planning will succeed between any two configurations that can be connected to the roadmaps produced. A simple breadth-first search algorithm can produce a sequence of edges that connect two nodes of the roadmap in a very short time, typically a small fraction of second in the machine used. Reconstructing the path is equally fast if no collisions need to be performed along recomputed local paths (see Section 3.2). In our implementation where collision checks are performed when recovering a local path, we spend a few tens of thousands of collision checks for connecting between different nodes in

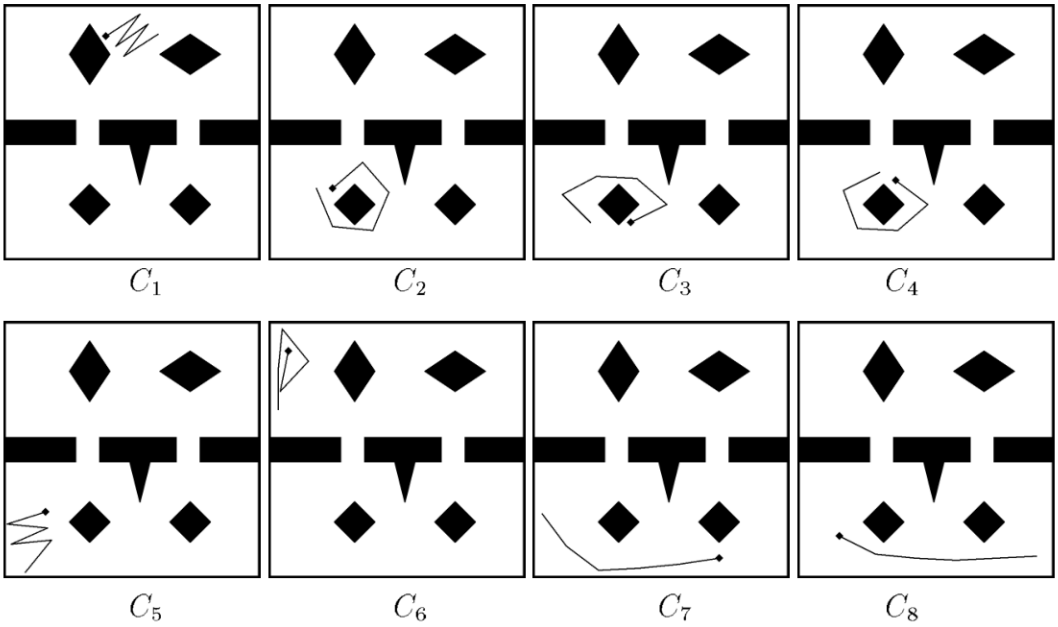


Figure 6: Scene 2, with 5-revolute-joint free-base robot (7-dof).

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks	Avg. nodes	Success Rate (%) C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
20.1	13.0	7.1	712661	541	96.7	6.7	6.7	6.7	6.7	86.7	6.7	10.0
30.1	19.6	10.5	1037739	924	100.0	16.7	16.7	16.7	16.7	90.0	16.7	16.7
40.1	26.0	14.1	1361134	1603	100.0	60.0	63.3	56.7	56.7	96.7	56.7	56.7
50.2	32.6	17.6	1674144	2460	100.0	93.3	93.3	93.3	93.3	100.0	96.7	93.3
60.3	39.2	21.1	1987967	2999	100.0	93.3	93.3	93.3	93.3	100.0	93.3	93.3
70.1	45.6	24.5	2336917	3695	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
80.4	52.3	28.1	2632712	4229	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Figure 7: Results with customized planner for scene of Fig. 6 (with expansion).

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks	Avg. nodes	Success Rate (%) C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
20.0	20.0	0.0	686580	527	96.7	3.3	3.3	3.3	3.3	86.7	3.3	3.3
30.0	30.0	0.0	987852	1005	100.0	30.0	30.0	26.7	30.0	96.7	30.0	30.0
40.3	40.3	0.0	1265245	1437	100.0	40.0	40.0	40.0	43.3	100.0	43.3	40.0
50.1	50.1	0.0	1534808	2238	100.0	80.0	80.0	76.7	76.7	100.0	76.7	76.7
60.0	60.0	0.0	1778678	2709	100.0	80.0	80.0	80.0	80.0	100.0	83.3	80.0
70.0	70.0	0.0	2058469	3384	100.0	90.0	90.0	90.0	90.0	100.0	90.0	90.0
80.2	80.2	0.0	2277226	4002	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Figure 8: Results with customized planner for scene of Fig. 6 (no expansion).

the roadmap. It is interesting to contrast the number of collision checks needed for learning and for query processing: collision checks for learning are 2 to 3 orders of magnitude larger than collision checks needed for answering queries. This is also true for any configurations we tried in the scene of Figure 2 and not only the eight configurations considered here. RPP, one of the few planners that can tackle the path planning problems arising from the configurations in Figure 2, takes a few tens of minutes on the average to solve these queries. Thus, even if learning time is included in the duration of the path planning process, our roadmap technique is still faster than RPP for this example. However, the above scene is very difficult for potential field methods. In simpler cases (see Section 6) RPP is equally fast, if not faster than our roadmap method.

Free-base articulated robot. We have performed the same experiments for a free-base articulated robot (see Figure 6). The robot has a total of 7 dof: 2 for its free base and 5 for its revolute joints. The parameter values of our planner are the same as in the previous experiments.

Figures 7 and 8 show the results obtained with and without expansion, respectively. 30 roadmaps were created independently for each row in the above tables. Again, in almost all cases, the percentage of successful connections to the roadmaps is greater with expansion than without (for the same total learning time). After a learning phase of 70 seconds, all configurations can be connected to the roadmaps produced. The actual number of collision checks for connecting $C_1, \ldots, C_8$ of Figure 6 to the roadmaps are again in the order of a few tens to a few thousands. This makes path planning between any two of the configurations shown in Figure 6 very fast: usually a fraction of a second in the machine used.

6 Results with general implementation

The customized implementation used in the previous section solves efficiently path planning problems involving planar articulated robots. In this section we demonstrate that the general implementation of the planner still gives very good results for a variety of examples.

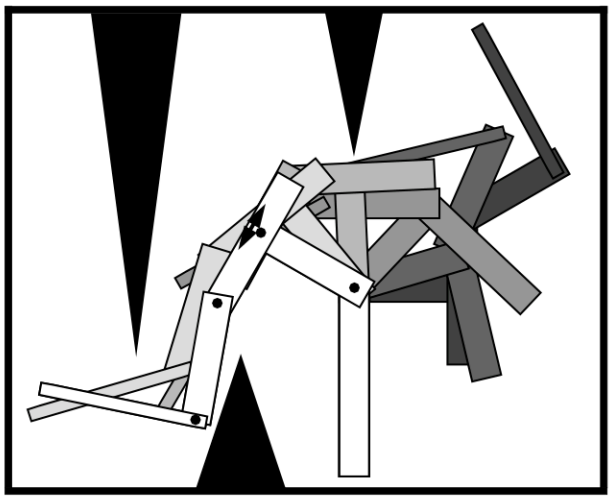


Figure 9: Scene 1, with 4-dof robot.

The planner considered here is essentially an implementation of the method described in Section 3. Unlike the customized implementation, this implementation does not use any specific techniques for local path planning, collision checking, or distance computation. Hence, as described in Section 3, the local path constructed between any two configurations is the straight line segment joining them in C-space; the distance function D is the one defined by Eqn. (1); and collision checking is done analytically, using routines from the PLAGEO library [Gie93]. The planner was implemented on a Silicon Graphics Indigo^2 workstation rated at 96.5 SPECfp92 and 90.4 SPECint92. This machine is comparable to the one we used for the results in the previous section. We report here on experimentation conducted with articulated robots with 4 or 5 joints connected by polygonal links. This general planner is directly applicable to robots with polyhedral links moving in 3D workspaces.

We present results obtained with two representative examples. In scene 1 in Figure 9, we have a 4-dof robot with three revolute joints and one prismatic joint (indicated by the double arrow). Scene 2 in Figure 10, is a slightly more difficult one, with a 5-revolute-joint robot and narrow areas in the workspace. For most existing planners, motion planning problems in both these scenes would be challenging ones. RPP is able to deal with these examples efficiently. Still, the cases treated here are considerably easier than in the scenes of Section 5, due to the relatively low number of dofs of the two robots, and the presence

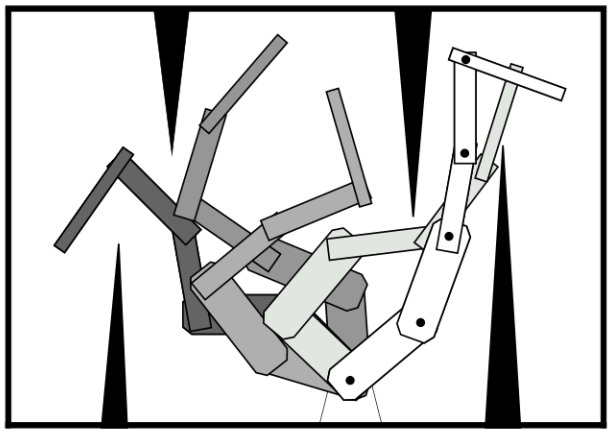


Figure 10: Scene 2, with 5-dof robot.

T_L (sec)	T_C (sec)	T_E (sec)	Coll. checks Learning: Scene 1	Success rate (%) Scene 1	Coll. checks Learning: Scene 2	Success rate (%) Scene 2
2.5	1.67	0.83	10078	53.3	9558	50
5	3.33	1.67	19756	93.3	18746	87
7.5	5	2.5	29525	100	27607	97
10	6.67	3.33	38607	100	36392	100

Figure 11: Results with general planner for scenes of Fig. 9 and 10.

of only few tight areas in the workspaces.

The experiments conducted with these two test scenes are similar to those in Section 5. For each scene, we consider only two "difficult" configurations s and g. Then, for a fixed construction time $\mathtt{T_C}$ and expansion time $\mathtt{T_E}$ (hence, a fixed learning time $\mathtt{T_L}$), we independently create 30 roadmaps. For each of these roadmaps we only consider its main connected component and we test whether the query with configurations (s, g) succeeds within 2.5 seconds. In other words, we test whether *both* s and g can be quickly connected to the main connected component of the roadmap with the method described in Section 3.2. We repeat this experiment for a number of different construction times $\mathtt{T_C}$ and expansion times $\mathtt{T_E}$, with $\mathtt{T_E} = \mathtt{T_C}/2$. For each such pair of times we report the success rate in answering the query (s, g).

The other parameters have the following fixed values, which are almost the same as in the experimentation reported in the previous section: `maxdist` $= 0.5$, `eps` $= 0.01$, `maxneighbors` $= 30$, $\mathtt{T_{RB_expand}} = 0.01$, $\mathtt{T_{RB_query}} = 0.05$ sec, and $\mathtt{N_{RB_query}} = 45$. Again, for the interpretation of the values for `maxdist` and `eps`, note that we scaled the two scenes in a way that the workspace obstacles just fit into the unit square.

In both Figures 9 and 10 the start configuration s is shown in dark grey, and the goal configuration g in white. In each figure, several robot configurations along a path solving the query are displayed using various grey levels. The results of the experiments described above are given in Figure 11. The average number of collision checks required to build the roadmaps is given in column 4 for scene 1 and in column 6 for scene 2. The query in scene 1 is solved in all 30 cases after having learned for 7.5 seconds. Learning for 5 seconds though suffices to successfully answer the query in more than 90% of the cases. In scene 2 we observe a similar behavior, although the required learning times are slightly higher.

These results show that the general implementation is able to efficiently solve rather complicated planning problems. However, when applied to problems involving more dofs, like those in the previous section, the learning times required to build good roadmaps are much longer. For example, experiments indicated that about 25 minutes of learning are required in order to obtain roadmaps that capture well the free C-space connectivity of the scene shown in Figure 2. Figure 12 reports some experimental results obtained over many

T_L (min)	T_C (min)	T_E (min)	Coll. checks	Success Rate (%) C_1	C_4	C_7	C_8
5	3.3	1.7	700000	80.0	30.0	30.0	43.3
10	6.7	3.3	1000000	96.7	76.7	70.0	53.3
15	10	5.0	1500000	96.7	80.0	73.3	90.0
20	13.3	6.7	1800000	100.0	96.7	96.7	100.0
25	16.7	8.3	2000000	100.0	100.0	100.0	100.0

Figure 12: Results with general planner for scene of Fig. 2 (with expansion).

independently constructed roadmaps, for different learning times. As in Section 5, we estimate the average number of collision checks needed during learning and the percentage of times that our planner succeeds in connecting some of the configurations of Figure 2 to the roadmap, over many independently constructed roadmaps, for different learning times. In such difficult cases, clearly, customization is desirable, if not necessary.

7 Conclusion

We have described a two-phase method to solve robot motion planning problems in static workspaces. In the learning phase, the method constructs a probabilistic roadmap as a collection of configurations randomly selected across the free C-space. In the query phase, it uses this roadmap to quickly process path planning queries, each specified by a pair of configurations. The learning phase includes a heuristic evaluator to identify difficult regions in the free C-space and increase the density of the roadmap in those regions. This feature enables us to construct roadmaps that capture well the connectivity of the free C-space.

The method is general and can be applied to virtually any type of holonomic robot. Furthermore, it can be easily customized to run more efficiently on some family of problems. Customization consists of replacing components of the general method, such as the local planner, by more specialized ones fitting better the characteristics of the considered scenes. In this paper, we have described techniques to customize the method to

planar articulated robots, and presented experimental results with both a general and a customized implementation of the method. The customized implementation can solve very difficult path planning queries involving many-dof robots in a fraction of a second, after a learning time of a few dozen seconds. The general implementation solves the same problems in several minutes, but it is still very efficient in less difficult problems.

In [KL94a, KL94b, Kav95, OŠ95] prior versions of the method have been applied to a great variety of holonomic robots including planar and spatial articulated robots with revolute, prismatic, and/or spherical joints, fixed or free base, and single or multiple kinematic chains. In [Šve93, ŠO94, ŠO95] a variation of the method (essentially one with a different general local planner) was also run successfully on examples involving nonholonomic car-like robots.

Experimental results show that our method can efficiently solve certain kinds of problems which are beyond the capabilities of other existing methods. For example, for planar articulated robots with many dofs, the customized implementation of Section 5 is much more consistent than the Randomized Path Planner (RPP) of [BL91]. Indeed, the latter can be very fast on some difficult problems, but it may also take prohibitive time on some others. We have not observed such disparity with our roadmap method. Moreover, after sufficient learning (usually on the order of a few dozen seconds), the probabilistic roadmap method answers queries considerably faster than RPP. However, when the learning time is included in the planning time, RPP is faster on many problems, since it does not perform any substantial precomputation.

An important question is how our method scales up when we consider scenes with more complicated geometry, since the cost of collision checking is much higher. First, let us note that in 2D workspaces the effect is likely to be limited if the bitmap collision-checking technique of Section 4 is used. Indeed, once bitmaps have been precomputed, collision checking is a constant-time operation; and the cost of computing bitmaps using the FFT-based technique described in [Kav93] only depends on the resolution (i.e., the size) of these bitmaps. However, more complicated geometry may require increasing the bitmap resolution in order to represent geometric details with desired accuracy. With 3D workspaces the situation is completely different, since we can no longer use the bitmap technique. Our experiments in 3D workspaces reported in [KL94b] show that the higher cost of collision checking increases the duration of the learning phase. It affects less the

query phase, since less collision checks are performed there. The results in [KL94b] also show that the duration of the learning phase remains quite reasonable (on the order of minutes), but they were obtained with simple 3D geometry (for example, the robot links were line segments). For more complicated geometries, the use of an iterative collision checker, like the one in [Qui94], will be advantageous. The collision checker in [Qui94] considers successive approximations of the objects and its running time, on the average, does not depend much on the geometric complexity of the scenes. RPP is another planner that heavily relies on collision checking. For long we ran RPP on geometrically simple problems; but, recently, we used it to automatically animate graphic 3D scenes of complex geometry [KKKL94] using the above iterative collision checker. We observed no dramatic slowdown of the RPP planner.

A challenging goal would now be to extend the method to dynamic scenes. One first question is: how should a roadmap computed for a given workspace be updated if a few obstacles are removed or added? The work in [BCH94, Ch92] discusses how to deal with changes in the environment in the context of the hybrid planner presented in [Ch92]. We hope that similar techniques could apply to our planner. Being able to plan when obstacles move will be very useful because then we could apply our method to scenes subject to small incremental changes. Such changes occur in many manufacturing (e.g., assembly) cells; while most of the geometry of such a cell is permanent and stationary, a few objects (e.g., fixtures) are added or removed between any two consecutive manufacturing operations. Similar incremental changes also occur in automatic graphic animation. A second question is: how should the learning and query phase be modified if some obstacles are moving along known trajectories? An answer to this question might consist of applying our roadmap method in the configuration$\times$time space of the robot [ELP86]. The roadmap would then have to be built as a directed graph, since local paths between any two nodes must monotonically progress along the time axis, with possibly additional constraints on their slope and curvature to reflect bounds on the robot's velocity and acceleration.

Acknowledgments: L.E. Kavraki and J.-C. Latombe were partially supported by ARPA grant N00014-92-J-1809 and ONR grant N00014-94-1-0721. L.E. Kavraki also acknowledges the support of the Rockwell Foundation. P. Švestka and M. Overmars were partially

supported by ESPRIT III BRA Project 6546 (PROMotion) and by the Dutch Organization for Scientific Research (NWO).

References

[ATBM92] J. M. Ahuactzin, E.-G. Talbi, P. Bessière, and E. Mazer. Using genetic algorithms for robot motion planning. *10th Europ. Conf. Artific. Intelligence.* 671–675. John Wiley and Sons, Ltd., London, England, 1992.

[BCH94] M. Barbehenn, P.C. Chen, and S. Hutchinson. An efficient hybrid planner in changing environments. *Proc. IEEE Int. Conf. Robotics and Automation*, 2755–2760, San Diego, CA, May 1994.

[BF94] J. Barraquand and P. Ferbach. Path planning through variational dynamic programming. *Proc. IEEE Int. Conf. Robotics and Automation*, 1839–1846, San Diego, CA, May 1994.

[BKLLMR95] B. Barraquand, L.E. Kavraki, J.-C. Latombe, T.-Y. Li, R. Motwani, and P. Raghavan. A random sampling scheme for robot path planning. *Robotics Research*, G. Giralt and G. Hirzinger (ed.), North Holland, 1996, to appear. (Proc. 7th Int. Symp. on Robotics Research, Herrsching, Germany, October 1995.)

[BLL92] J. Barraquand, B. Langlois, and J.-C. Latombe. Numerical potential field techniques for robot path planning. *IEEE Tr. Syst., Man, and Cybern.*, 22(2):224–241, 1992.

[BL91] J. Barraquand and J.-C. Latombe. Robot motion planning: a distributed representation approach. *Int. J. Robotics Research*, 10:628–649, 1991.

[BG94] S. Berchtold and B. Glavina. A scalable optimizer for automatically generated manipulator motions. *Proc. IEEE/RSJ/GI Int. Conf. Intelligent Robots and Systems*, 1796-1802, München, Germany, 1994.

[Can88] J.F. Canny. *The complexity of robot motion planning.* MIT Press, Cambridge, MA, 1988.

[CL90] J.F. Canny and M.C. Lin. An opportunistic global path planner. *Proc. IEEE Int. Conf. Robotics and Automation*, 1554–1559, Cincinnati, OH, 1990.

[CG93] D. Chalou and M. Gini. Parallel robot motion planning. *Proc. of IEEE Int. Conf. Robotics and Automation*, 24–51, Atlanta, GA, 1993.

[CL95] H. Chang and T.-Y. Li. Assembly maintainability study with motion planning. *Proc. IEEE Int. Conf. Robotics and Automation*, Nayoga, JP, 1995.

[Ch92] P.C. Chen. Improving path planning with learning. *Proc. Machine Learning Conference*, 55–61, 1992.

[Ch93] P.C. Chen. *Adaptive path planning in changing environments.* Report SAND92-2744, Sandia National Laboratories, 1993.

[CH92] P.C. Chen and Y.K. Hwang. SANDROS: A motion planner with performance proportional to task difficulty. *Proc. of IEEE Int. Conf. Robotics and Automation,* 2346-2353, Nice, France, 1992.

[CB94] H. Choset and J. Burdick. Sensor based planning and nonsmooth analysis. *Proc. of IEEE Int. Conf. Robotics and Automation*, 3034-3041, San Diego, CA, 1994.

[ELP86] M. Erdmann and T. Lozano-Pérez. On multiple moving objects. *Proc. IEEE Int. Conf. Robotics and Automation*, 1152–1159, 1986.

[FT87] B. Faverjon and P. Tournassoud. A local approach for path planning of manipulators with a high number of degrees of freedom. *Proc. IEEE Int. Conf. Robotics and Automation*, 1152–1159, Raleigh, NC, 1987.

[FT90] B. Faverjon and P. Tournassoud. A practical approach to motion planning for manipulators with many degrees of freedom. *Robotics Research 5*, H. Miura and S. Arimoto (Eds.), 65–73, MIT Press, MA, 1990.

[Gie93] G.-J. Giezeman. *PlaGeo—A library for planar geometry.* Tech. Rep., Dept. Comput. Sci., Utrecht Univ., Utrecht, The Netherlands, 1993.

[GMKL92] L. Graux, P. Millies, P.L. Kociemba, and B. Langlois. Integration of a path generation algorithm into off-line programming of airbus panels. *Aerospace Automated Fastening Conf. and Exp.*, SAE Tech. Paper 922404, October 1992.

[GG92] K. Gupta and Z. Gou. Sequential search with backtracking. *Proc. of IEEE Int. Conf. Robotics and Automation*, 2328–2333, Nice, France, 1992.

[GZ94] K. Gupta and X. Zhu. Practical motion planning for many degrees of freedom: A novel approach within sequential framework. *Proc. of IEEE Int. Conf. Robotics and Automation*, 2038–2043, San Diego, CA, 1994.

[HST94] Th. Horsch, F. Schwarz, and H. Tolle. Motion planning for many degrees of freedom - random reflections at C-space obstacles. *Proc. IEEE Int. Conf. Robotics and Automation*, 3318–3323, San Diego, CA, 1994.

[Kav93] L.E. Kavraki. Computation of configuration-space obstacles using the fast fourier transform. *Proc. IEEE Int. Conf. Robotics and Automation*, 255–261, Atlanta, GA, 1993. To appear in *IEEE Tr. Robotics and Automation.*

[Kav95] L.E. Kavraki. *Random networks in configuration space for fast path planning*, Ph.D. Thesis, Tech. Rep. STAN-CS-95-1535, Dept. Comput. Sci., Stanford Univ., Stanford, CA, January 1995.

[KL93] L.E. Kavraki and J.-C. Latombe. *Randomized preprocessing of configuration space for fast path planning.* Tech. Rep. STAN-CS-93-1490, Dept. Comput. Sci., Stanford Univ., Stanford, CA, September 1993.

[KL94a] L.E. Kavraki and J.-C. Latombe. Randomized preprocessing of configuration space for fast path planning. *Proc. IEEE Int. Conf. Robotics and Automation*, 2138–2145, San Diego, CA, 1994.

[KL94b] L.E. Kavraki and J.-C. Latombe. Randomized preprocessing of configuration space for path planning: Articulated robots. *Proc. IEEE/RSJ/GI Int. Conf. Intelligent Robots and Systems*, 1764-1772, Germany, 1994.

[KLMR95] L.E. Kavraki, J.-C. Latombe, R. Motwani, and P. Raghavan. Randomized query processing in robot path planning. *Proceedings of the 27th Annual*

ACM Symposium on Theory of Computing (STOC), 353-362, Las Vegas, NV, May 1995,

[Kod87] D.E. Koditschek. Exact robot navigation by means of potential functions: some topological considerations. *Proc. IEEE Int. Conf. Robotics and Automation*, 1-6, Raleigh, NC, 1987.

[KKKL94] Y. Koga, K. Kondo, J. Kuffner, and J.-C. Latombe. Planning motions with intentions. *Proc. of SIGGRAPH'94*, 395-408, FL, 1994.

[Kon91] K. Kondo. Motion planning with six degrees of freedom by multistrategic bidirectional heuristic free-space enumeration. *IEEE Tr. on Robotics and Automation*, 7(3):267–277, 1991.

[Lat91] J.-C. Latombe. *Robot motion planning.* Kluwer Academic Publishers, Boston, MA, 1991.

[LTJ90] J.-P. Laumond, M. Taix, and P. Jacobs. A motion planner for car-like robots based on a global/local approach. Proc. IEEE Internat. Workshop Intell. Robot Syst. 765–773. 1990.

[LRDG90] J. Lengyel, M. Reichert, B.R. Donald, and P. Greenberg. Real-time robot motion planning using rasterizing computer graphics hardware. *Proc. SIGGRAPH'90*, 327–335, Dallas, TX, 1990.

[LP83] T. Lozano-Pérez. Spatial planning: a configuration space approach. *IEEE Tr. on Computers*, 32:108–120, 1983.

[LPO91] T. Lozano-Pérez and P. O'Donnel. Parallel robot motion planning. *Proc. IEEE Int. Conf. Rob. and Automation*, 1000–1007, Sacramento, CA, 1991.

[LPW79] T. Lozano-Pérez and M.A. Wesley. An algorithm for planning collision-free paths among polyhedral obstacles. *Comm. of the ACM*, 22(10):560-570, 1979.

[Mas92] J. Mastwijk. *Motion planning using potential field methods.* Master Thesis, Dept. Comput. Sci., Utrecht Univ., Utrecht, The Netherlands, August 1992.

[OY82] C. Ó'Dúnlaing and C.K. Yap. A retraction method for planning the motion of a disc. *J. of Algorithms*, 6:104-111, 1982.

[Ove92] M. Overmars. *A random approach to motion planning.* Tech. Rep. RUU-CS-92-32, Dept. Comput. Sci., Utrecht Univ., Utrecht, The Netherlands, October 1992.

[OŠ95] M. Overmars and P. Švestka. A probabilistic learning approach to motion planning. In *Algorithmic Foundations of Robotics*, K. Goldberg et al. (eds.), A K Peters, Wellesley, MA, 19-37, 1995.

[Qui94] S. Quinlan. Efficient distance computation between non-convex objects. *Proc. IEEE Int. Conf. Robotics and Automation*, 3324-3330, San Diego, CA, 1994.

[RC94] E. Rimon and J.F. Canny. Construction of C-space roadmaps from local sensory data, What should the sensors look for? *Proc. IEEE Int. Conf. Robotics and Automation*, 117-123, San Diego, CA, 1994.

[RK92] E. Rimon and D.E. Koditschek. Exact robot navigation using artificial potential functions. *IEEE Tans. on Robotics and Automation*, 8:501-518, 1992.

[Šve93] P. Švestka. *A probabilistic approach to motion planning for car-like robots.* Tech. Rep. RUU-CS-93-18, Dept. Comput. Sci., Utrecht Univ., Utrecht, The Netherlands, April 1993.

[ŠO94] P. Švestka and M. Overmars. *Motion planning for car-like robots using a probabilistic learning approach.* To appear in *Int. Journal of Robotics Research*, 1995.

[ŠO95] P. Švestka and M. Overmars. Coordinated motion planning for multiple car-like robots using probabilistic roadmaps. *Proc. IEEE Int. Conf. Robotics and Automation*, Nagoya, Japan, 1631-1636, 1995.

[ZG93] X. Zhu and K. Gupta. *On local minima and random search in robot motion planning.* Unpublished Tech. Report, Simon Fraser Univ., BC, Canada, 1993 (personal communication).

8 Biographies

Lydia Kavraki received her B.S. degree in Computer Science from the University of Crete, Greece, in 1989 and her M.S. and Ph.D. degrees in Computer Science from Stanford University in 1992 and 1995 respectively. She is currently a Research Associate at the Robotics Laboratory of Stanford University. Her research interests include motion planning, assembly planning, geometric computing and their applications in areas of molecular biology such as pharmaceutical drug design. Her work emphasizes the use of probabilistic techniques for solving complex geometric problems in moderately high dimensions.

Petr Švestka was born in Prague, Czechoslovakia, on October 17, 1968. He received a degree in computer science from the University of Utrecht, Utrecht, the Netherlands, in 1993. Currently he is working toward his Ph.D. degree, also at Utrecht University. His research activities include probabilistic path planning for various types of robots, as well as multi-robot path planning.

Jean-Claude Latombe received the B.S. and M.S. degrees in electrical engineering and the Ph.D. degree in computer science from the National Polytechnic Institute of Grenoble, in 1969, 1972, and 1977, respectively. He is currently a Professor of Computer Science at Stanford University, where he is also the director of the Computer Science Robotics Laboratory. From 1980 to 1984, he was a faculty member at Ecole Nationale Superieure d'Informatique et de Mathematiques Appliquees de Grenoble (ENSIMAG). From 1984 to 1987, he was the executive president of Industry and Technology for Machine Intelligence (ITMI), a company that he cofounded in 1982 for commercializing robot systems and expert systems. His current research interests lie mainly in geometric computing and motion planning, and in their applications to manufacturing, mobile robot navigation, concurrent design, graphic animation, rationale drug design, and computer-assisted surgery.

Mark Overmars received his PhD degree in Computer Science in 1983 from Utrecht University in the Netherlands. Currently he is a full professor in Computer Science at the same university. His main research interests include Computational Geometry and its application in areas like Computer Graphics and Robotics. In Robotics his work concentrates on exact and heuristic methods for motion planning and on algorithmic issues in robotic manipulation.